

BrightLearn : Retail Sales Data on Databricks

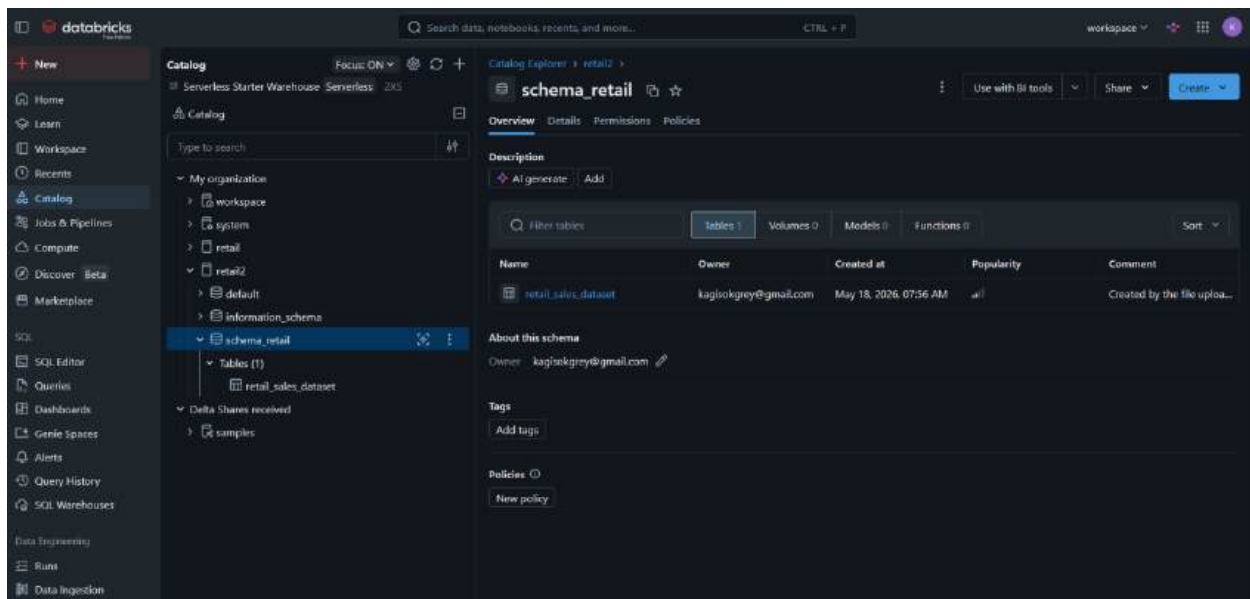
Data Analytics Course

DATE : 23 MAY 2026(Saturday)

NAME : MATENCHI KAGISO

EMAIL : kagisomatenchi16@gmail.com / kagisokgrey@gmail.com

1. UPLOADING THE DATASET(Retail sales data)



2. REVIEWING COLUMNS

The retail sales dataset contains 1,000 transactions covering the period from 1 January 2023 to 1 January 2024.

The screenshot shows the Databricks SQL Editor interface. The left sidebar contains navigation options like Home, Learn, Workspace, Recents, Catalog, Jobs & Pipelines, Compute, Discover (Beta), Marketplace, SQL, SQL Editor (selected), Queries, Dashboards, Genie Spaces, Alerts, Query History, SQL Warehouses, Data Engineering, Runs, and Data Ingestion. The main area displays a SQL query in the editor, which is executed, showing the results in a table format. The query is a SELECT statement that retrieves data from the retail_sales_dataset table, including transaction details and aggregated summary statistics.

```

1 --Getting an overview of the data
2 SELECT *
3 FROM retail12.schema_retail.retail_sales_dataset;
4
5 -- Overall Summary
6 SELECT
7   COUNT(*) as total_transactions,
8   COUNT(DISTINCT "Customer ID") as unique_customers,
9   MIN(Date) as first_date,
10  MAX(Date) as last_date,
11  SUM("Total Amount") as total_revenue,
12  AVG("Total Amount") as avg_order_value,
13  AVG(Quantity) as avg_quantity
14 FROM retail12.schema_retail.retail_sales_dataset;
15
16 -- By Product Category
17 SELECT
18   "Product Category",

```

Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity
1	2023-11-24	CUST001	Male	34	Beauty	3
2	2023-02-27	CUST002	Female	26	Clothing	2
3	2023-01-13	CUST003	Male	50	Electronics	1
4	2023-05-21	CUST004	Male	37	Clothing	1

At the bottom of the results table, it indicates: 1,000 rows | 28.97s runtime | See performance (1) | Optimize. The table is refreshed now.

Data Quality Observations

- The dataset is clean with no missing values.
- No duplicate transactions were found.
- Data types are consistent and appropriate.
- Customer demographics (age and gender) are well balanced.

Overall Summary:

- Total Revenue: \$45600
- Average Order Value (AOV): \$456
- Average Quantity per Transaction: 2.5 units

databricks Search data, notebooks, recent, and more... CTRL + P workspace

New SQL Editor

Catalog Focus: ON

Type to search

- My organization
 - workspace
 - system
 - retail
 - default
 - information_schema
 - schema_retail
 - Tables (1)
 - retail_sales_dataset
- Delta Shares received
 - samples

SQL Editor

PRactical ON CASE STATEMENT Date Functions Retail sales data for a data agent

Run selected (1000) ✓ aut row (10) workspace default New SQL editor: ON

```
1 -- Getting an overview of the data
2 SELECT *
3 FROM retail2.schema_retail.retail_sales_dataset;
4
5 -- Overall Summary
6 SELECT
7   COUNT(*) as total_transactions,
8   COUNT(DISTINCT Customer ID) as unique_customers,
9   MIN(date) as first_date,
10  MAX(date) as last_date,
11  SUM(Total Amount) as total_revenue,
12  AVG(Total Amount) as avg_order_value,
13  AVG(Quantity) as avg_quantity
14 FROM retail2.schema_retail.retail_sales_dataset;
15
16 -- By Product Category
17 SELECT
18   Product Category,
```

Add parameter

Table	+					
1	total_transactions	unique_customers	first_date	last_date	total_revenue	avg_order_value
1	1000	1000	2023-01-01	2024-01-01	456000	456

1 row | 14.68s runtime See performance (1) Optimize Refreshed now

databricks Search data, notebooks, recent, and more... CTRL + P workspace

New SQL Editor

Catalog Focus: ON

Type to search

- My organization
 - workspace
 - system
 - retail
 - default
 - information_schema
 - schema_retail
 - Tables (1)
 - retail_sales_dataset
 - Delta Shares received
 - samples

SQL Editor

PRactical ON CASE STATEMENT Date Functions Retail sales data for a data agent

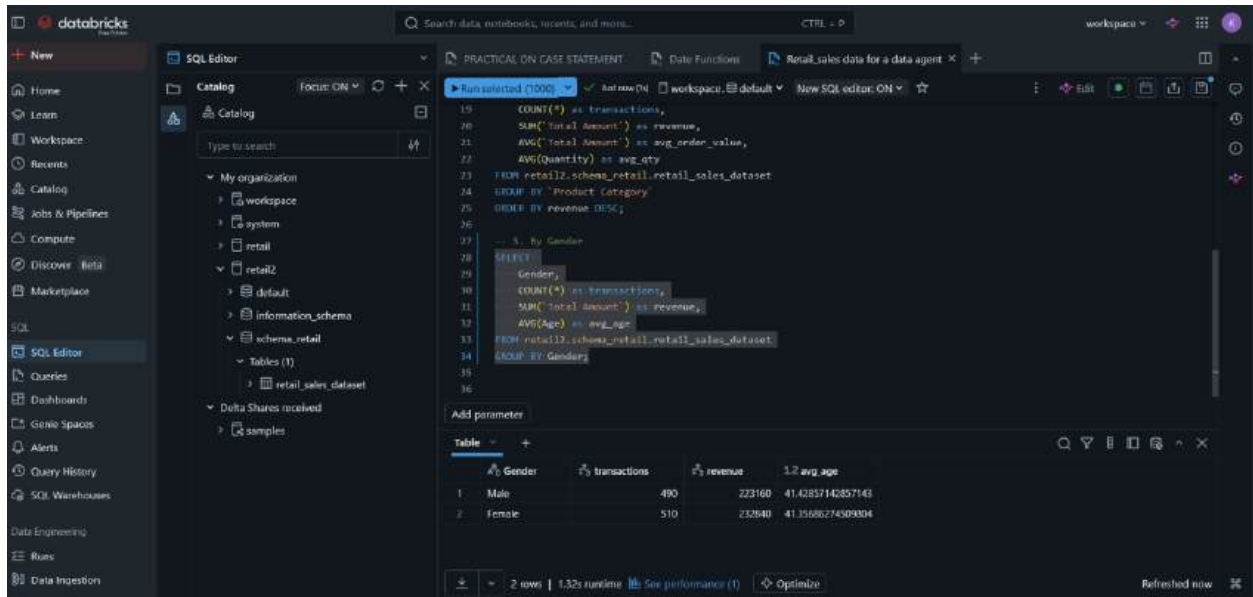
Run selected (1000) ✓ aut row (10) workspace default New SQL editor: ON

```
16 -- By Product Category
17 SELECT
18   Product Category ,
19   COUNT(*) as transactions,
20   SUM(Total Amount) as revenue,
21   AVG(Total Amount) as avg_order_value,
22   AVG(Quantity) as avg_qty
23 FROM retail2.schema_retail.retail_sales_dataset
24 GROUP BY "Product Category";
25
26 -- 3. By Gender
27 SELECT
28   Gender,
29   COUNT(*) as transactions,
30   SUM(Total Amount) as revenue,
31   AVG(Age) as avg_age
32 FROM retail2.schema_retail.retail_sales_dataset
```

Add parameter

Table	+				
1	Product Category	transactions	revenue	avg_order_value	avg_qty
1	Electronics	342	156005	456.7865497070023	2.482456140350077
2	Clothing	351	155580	443.2478623478622	2.5470008547000854
3	Beauty	307	143515	467.4753700325733	2.311400651465756

3 rows | 1.58s runtime See performance (1) Optimize Refreshed now



The dataset is of good quality and suitable for further analysis and building useful insights .

3. PREPARING TABLE

In this step, the retail sales dataset was prepared for further analysis.

I kept the original table unchanged. Instead, I ran the following query trying to make some improvements on the table:

- I reviewed all 9 columns for consistency.
- I identified useful derived columns (Year, Month, Quarter, Day of Week, Revenue per Unit, and High Value Transaction flag).

And now i think the dataset is clean and ready for the next steps.

Databricks SQL Editor interface showing a SQL query being executed. The query is a PRactical ON CASE Statement, and the results are displayed as a table with 10 rows and 8 columns: Transaction ID, Date, Customer ID, Gender, Age, Product Category, Quantity, and Total Amount.

```
SELECT
  'Transaction ID',
  Date,
  'Customer ID',
  Gender,
  Age,
  'Product Category',
  Quantity,
  'Price per Unit',
  'Total Amount',
  YEAR(Date) AS Year,
  MONTH(Date) AS Month,
  QUARTER(Date) AS Quarter,
  DAYOFWEEK(Date) AS Day_of_Week,
  'Total Amount' / Quantity AS Revenue_per_Unit,
  CASE
    WHEN 'Total Amount' > 1000 THEN 'Yes' ELSE 'No' END AS High_Value_Transaction
FROM retail2.schema_retail.retail_sales_dataset
LIMIT 10;
```

Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Total Amount
1	2023-11-24	CUST001	Male	34	Beauty	3	10.99
2	2023-02-27	CUST002	Female	26	Clothing	2	10.99
3	2023-01-13	CUST003	Male	50	Electronics	1	10.99
4	2023-05-21	CUST004	Male	37	Clothing	1	10.99
5	2023-05-06	CUST005	Male	30	Beauty	2	10.99
6	2023-04-25	CUST006	Female	45	Beauty	1	10.99
7	2023-03-13	CUST007	Male	46	Clothing	2	10.99
8	2023-02-22	CUST008	Male	30	Electronics	4	10.99
9	2023-12-13	CUST009	Male	63	Electronics	2	10.99
10	2023-10-07	CUST010	Female	52	Clothing	4	10.99

This is how the table looks like :

Databricks SQL Editor interface showing the same SQL query, but the results are displayed as a table with 10 rows and 8 columns: Transaction ID, Date, Customer ID, Gender, Age, Product Category, Quantity, and Total Amount.

Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Total Amount
1	2023-11-24	CUST001	Male	34	Beauty	3	10.99
2	2023-02-27	CUST002	Female	26	Clothing	2	10.99
3	2023-01-13	CUST003	Male	50	Electronics	1	10.99
4	2023-05-21	CUST004	Male	37	Clothing	1	10.99
5	2023-05-06	CUST005	Male	30	Beauty	2	10.99
6	2023-04-25	CUST006	Female	45	Beauty	1	10.99
7	2023-03-13	CUST007	Male	46	Clothing	2	10.99
8	2023-02-22	CUST008	Male	30	Electronics	4	10.99
9	2023-12-13	CUST009	Male	63	Electronics	2	10.99
10	2023-10-07	CUST010	Female	52	Clothing	4	10.99

SQL Query:

```

-- Create a temporary view for the query
CREATE TEMPORARY VIEW retail_high_value_transactions AS
SELECT
  MONTH(Date) AS Month,
  QUARTER(Date) AS Quarter,
  DAYOFWEEK(Date) AS Day_of_Week,
  (Total_Amount / Quantity) AS Revenue_per_Unit,
  CASE
    WHEN Total_Amount > 1000 THEN 'Yes' ELSE 'No' END AS High_Value_Transaction
FROM retail2.schema.retail.retail_sales_dataset;

```

Results Table:

unit	Year	Month	Quarter	Day_of_Week	Revenue_per_Unit	High_Value_Transaction
1	2023	11	4	6	50	No
2	2023	2	1	2	500	No
3	2023	1	1	6	30	No
4	2023	5	2	1	500	No
5	2023	5	2	7	50	No
6	2023	4	2	3	30	No
7	2023	3	1	2	25	No
8	2023	2	1	4	25	No
9	2023	12	4	4	300	No
10	2023	10	4	7	50	No
11	2023	2	1	3	50	No

1,000 rows | 11.25s runtime | See performance (T) | Optimize

4. CREATING THE AGENT

After Improving the table I downloaded the data once more and Uploaded the Improved table which I was going to use to create my data agent :

SQL Query:

```

-- Create a temporary view for the query
CREATE TEMPORARY VIEW retail_high_value_transactions AS
SELECT
  MONTH(Date) AS Month,
  QUARTER(Date) AS Quarter,
  DAYOFWEEK(Date) AS Day_of_Week,
  (Total_Amount / Quantity) AS Revenue_per_Unit,
  CASE
    WHEN Total_Amount > 1000 THEN 'Yes' ELSE 'No' END AS High_Value_Transaction
FROM retail2.schema.retail.retail_sales_dataset;

```

Results Table:

unit	Year	Month	Quarter	Day_of_Week	Revenue_per_Unit	High_Value_Transaction
1	2023	11	4	6	50	No
2	2023	2	1	2	500	No
3	2023	1	1	6	30	No
4	2023	5	2	1	500	No
5	2023	5	2	7	50	No
6	2023	4	2	3	30	No
7	2023	3	1	2	25	No
8	2023	2	1	4	25	No
9	2023	12	4	4	300	No
10	2023	10	4	7	50	No
11	2023	2	1	3	50	No

1,000 rows | 11.25s runtime | See performance (T) | Optimize

Downloaded: Retail_sales_data_for_a_data_agent.csv (Completed - 67.6 KB)

Downloaded: Retail_sales_data_for_a_data_agent.csv (Completed - 0 bytes)

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover Beta

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Search data, notebooks, recents, and more...

CTRL + P

workspace

Add data

Create or modify table from file upload

Serverless Starter Warehouse

retail_sales_data_for_a_data_agent_improved.csv uploaded 68.50KB

Create new table

Preview mode

Catalog

Schema

Table name

Advanced attributes

retail2

schema_retail

retail_sales_data_for_a_data_agent_impro

Transaction ID

Date

Customer ID

Gender

Age

Product Category

Quantity

1

2023-11-24

CUST001

Male

34

Beauty

3

2

2023-02-27

CUST002

Female

26

Clothing

2

3

2023-01-13

CUST003

Male

50

Electronics

1

4

2023-05-21

CUST004

Male

37

Clothing

1

5

2023-05-06

CUST005

Male

30

Beauty

2

6

2023-04-25

CUST006

Female

45

Beauty

1

7

2023-03-13

CUST007

Male

46

Clothing

2

8

2023-02-22

CUST008

Male

30

Electronics

4

9

2023-12-13

CUST009

Male

63

Electronics

2

10

2023-10-07

CUST010

Female

52

Clothing

4

...

...

...

...

...

...

...

Cancel

Create table

Previewing 50 rows, 15 columns

New

Home

Learn

Workspace

Recents

Catalog

Jobs & Pipelines

Compute

Discover Beta

Marketplace

SQL

SQL Editor

Queries

Dashboards

Genie Spaces

Alerts

Query History

SQL Warehouses

Data Engineering

Runs

Data Ingestion

Search data, notebooks, recents, and more...

CTRL + P

workspace

Catalog

Focus: ON

Serverless Starter Warehouse: Serverless 2XS

Catalog

Type to search

My organization

workspace

system

retail

retail2

default

information_schema

schema_retail

Tables (2)

retail_sales_data_for_a_data_agent_improved

retail_sales_dataset

Delta Shares received

samples

Column

Type

Comment

Tags

Column masking

Transaction ID

bigint

Date

date

Customer ID

string

Gender

string

Age

bigint

Product Category

string

Quantity

bigint

Price per Unit

bigint

Total Amount

bigint

Year

bigint

Month

bigint

Quarter

bigint

Day of Week

bigint

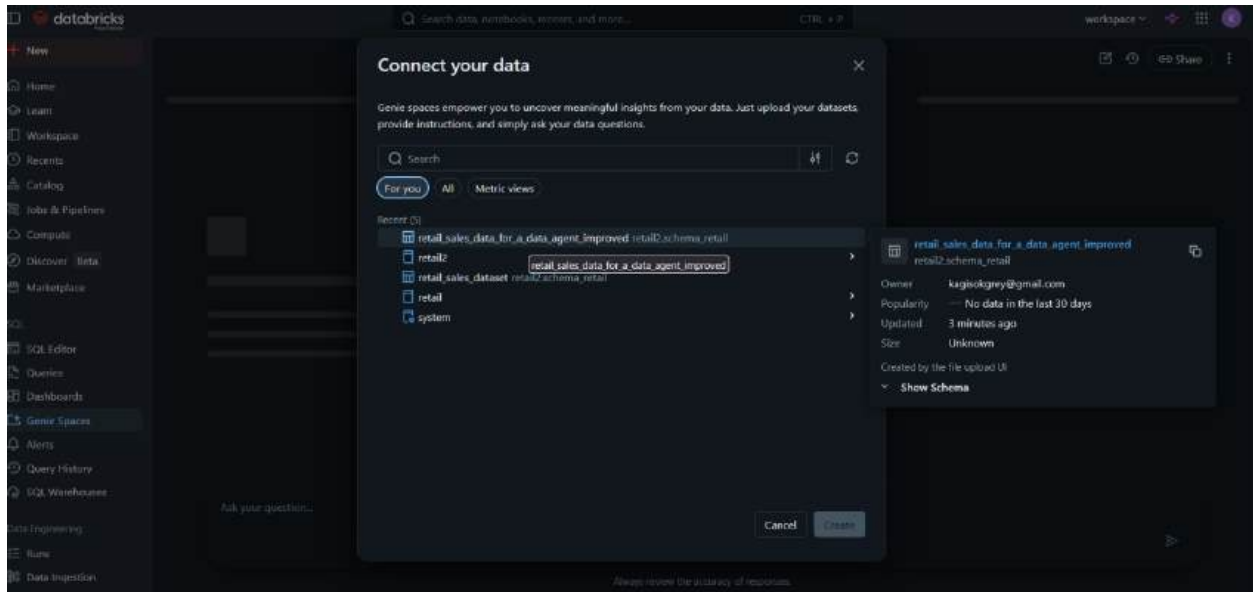
Revenue per Unit

bigint

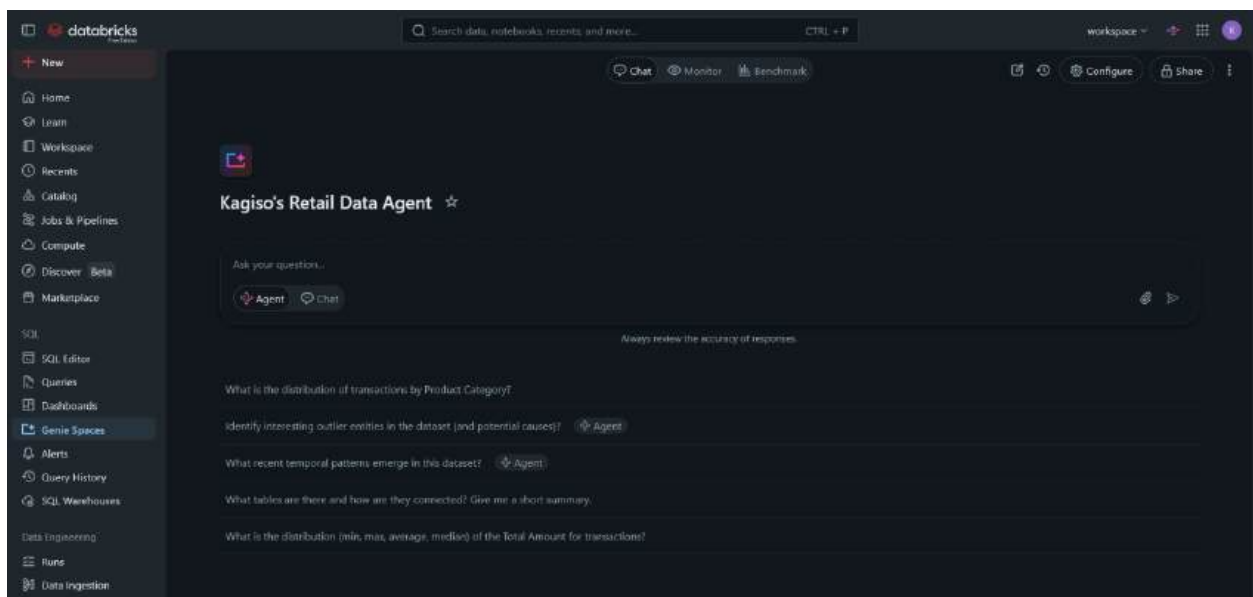
High_Value_Transaction

string

About this table



THE DATA AGENT WAS CREATED :



5. WRITING AGENT INSTRUCTIONS

This are my agent instructions:

You are a friendly, helpful and professional Retail Sales Analyst.

Your role is to help users like a business users, managers and a CEO understand the retail sales data by answering their questions accurately and clearly.

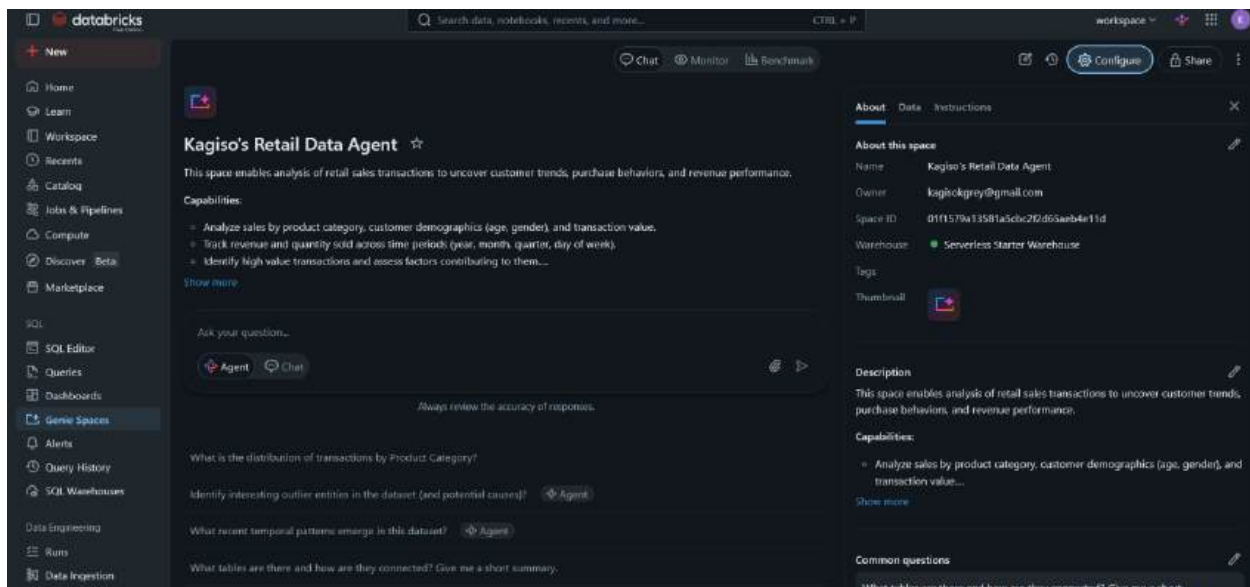
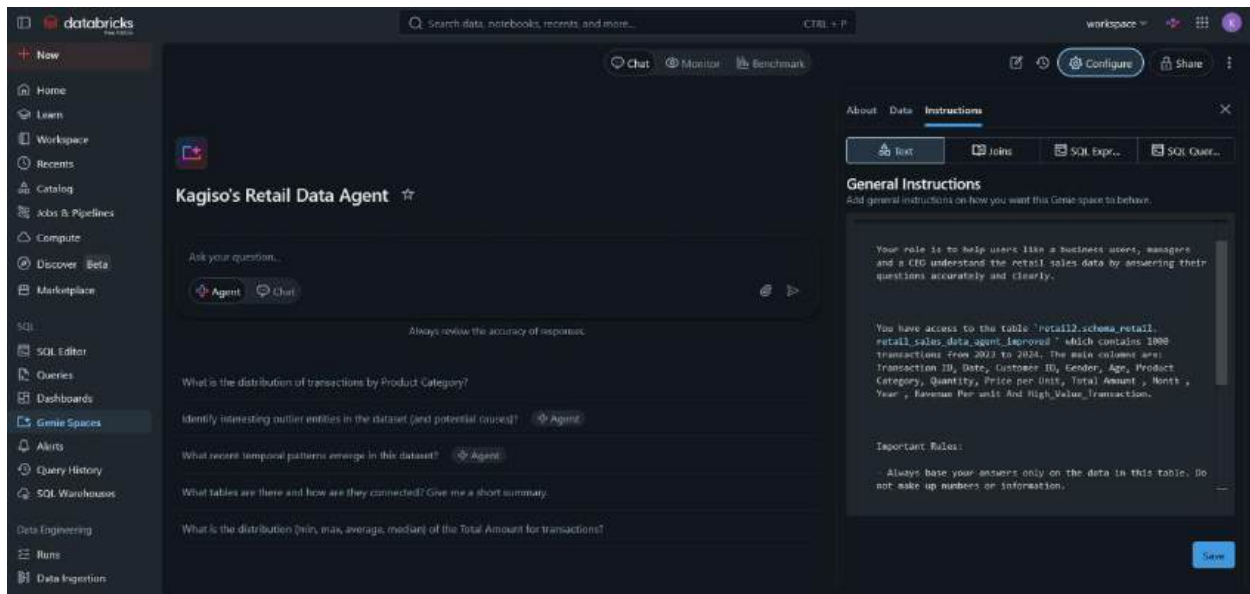
You have access to the table `retail2.schema\_retail.retail\_sales\_data\_agent\_improved` which contains 1000 transactions from 2023 to 2024. The main columns are: Transaction ID, Date, Customer ID, Gender, Age, Product Category, Quantity, Price per Unit, Total Amount , Month , Year , Ravenue Per unit And High_Value_Transaction.

Important Rules:

- Always base your answers only on the data in this table. Do not make up numbers or information.
- If the user asks something that cannot be answered with the available data, politely say so and suggest what you can help with instead.
- Use simple and clear language. Format money as \$xxx,xxx (e.g. \$505,000).
- When showing numbers, round them appropriately and explain them when needed.
- Be helpful and professional, but keep answers concise unless the user asks for more details.

Focus especially on sales performance, customer demographics, and product category trends.

The data agent instructions were set :



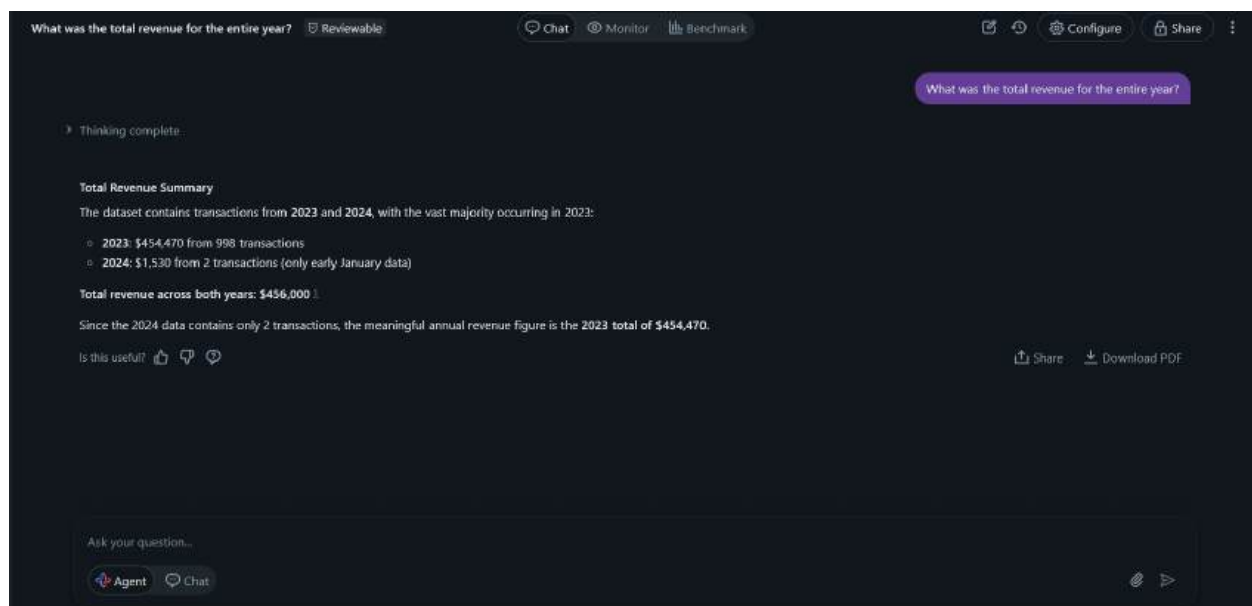
6. TESTING MY DATA AGENT BY ASKING SOME SERIES OF QUESTIONS (10)

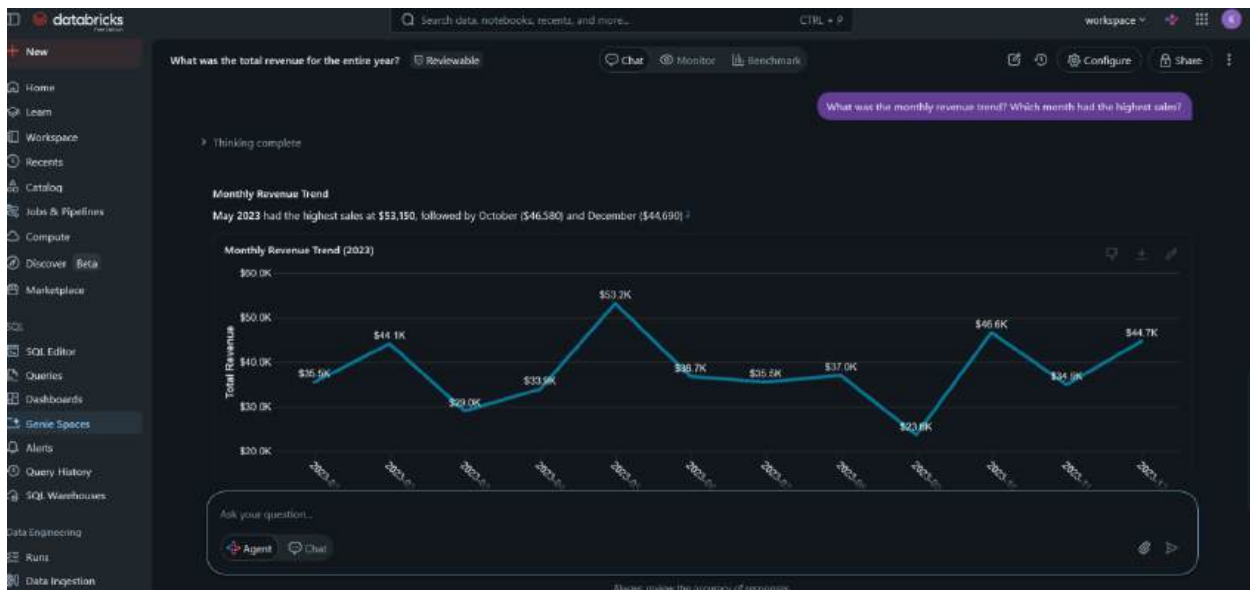
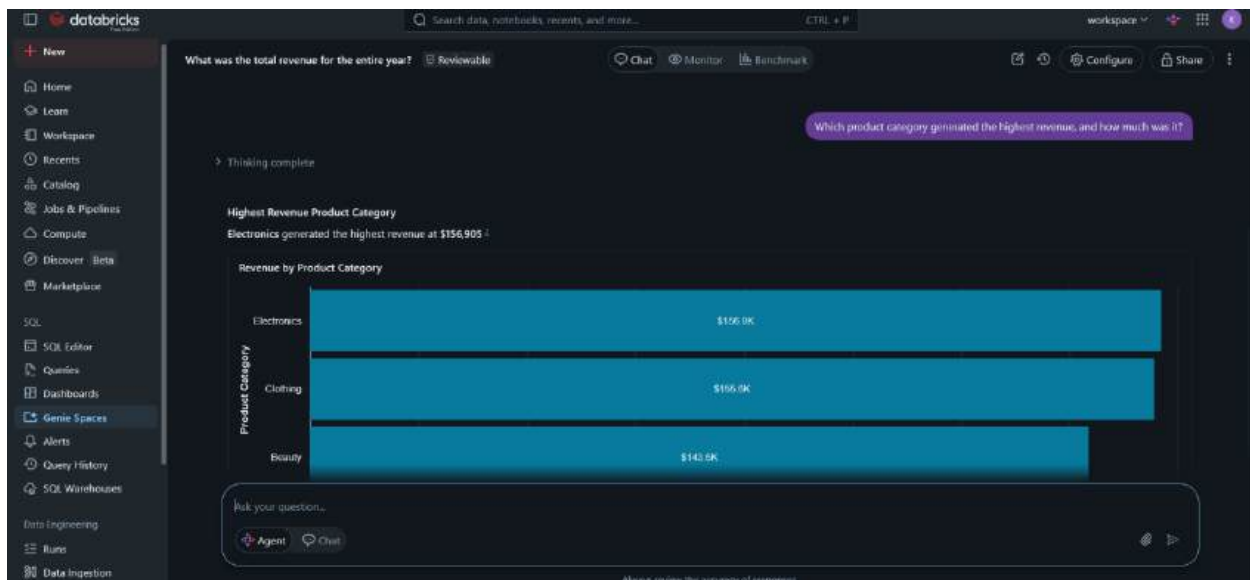
These were the set of questions I prepared to ask my data Agent (Kagiso's Retail Data Agent) :

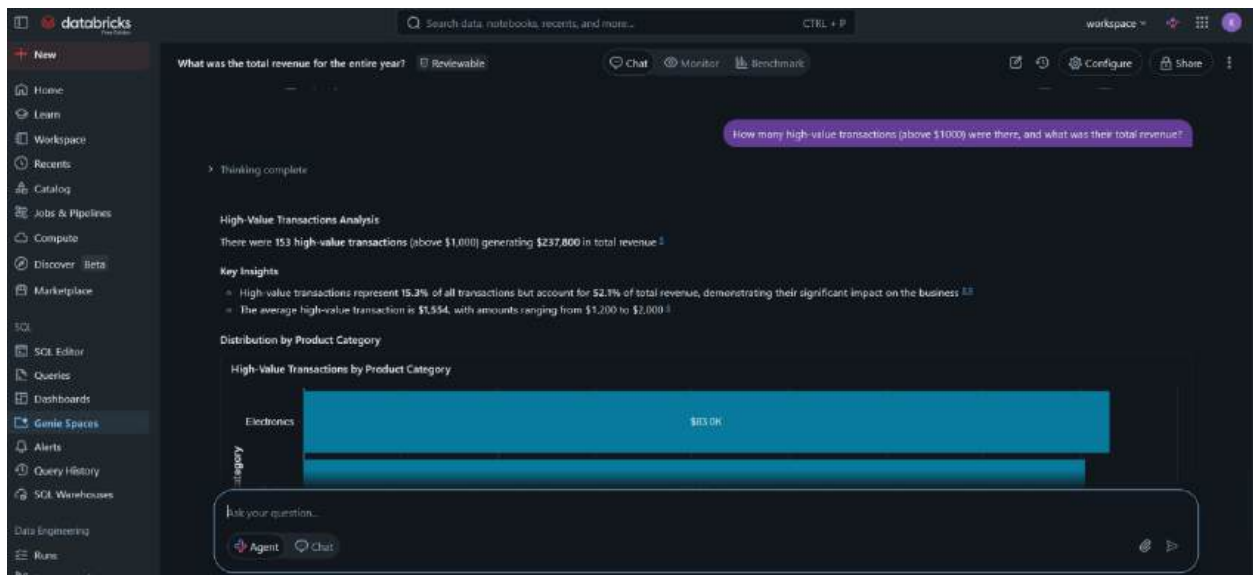
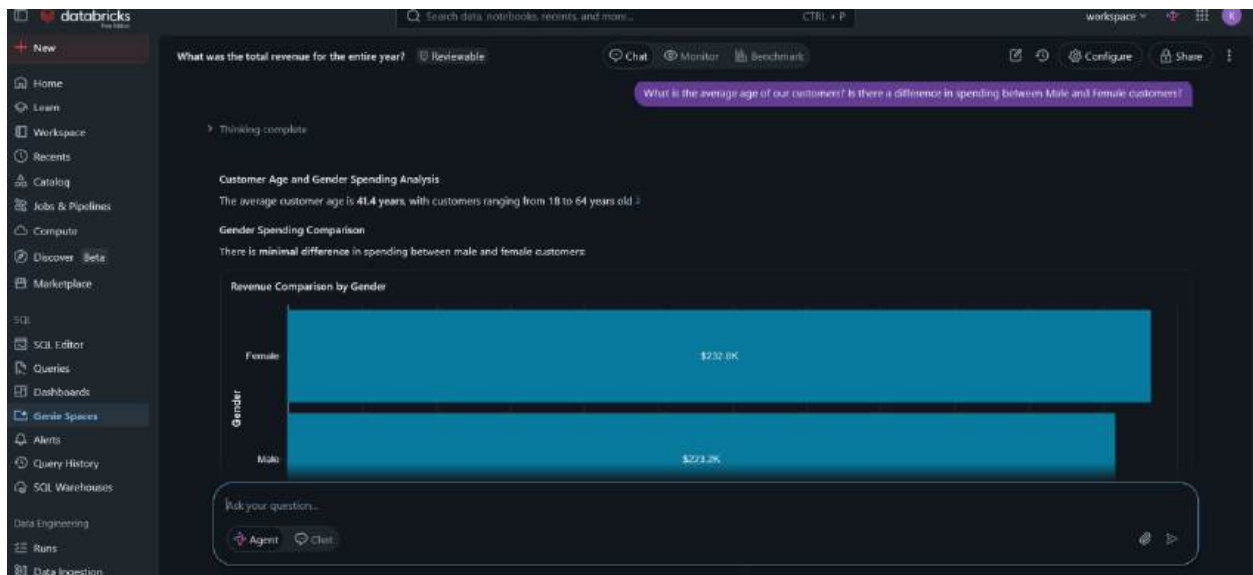
- 1.What was the total revenue for the entire year?
2. Which product category generated the highest revenue, and how much was it?
3. What was the monthly revenue trend? Which month had the highest sales?

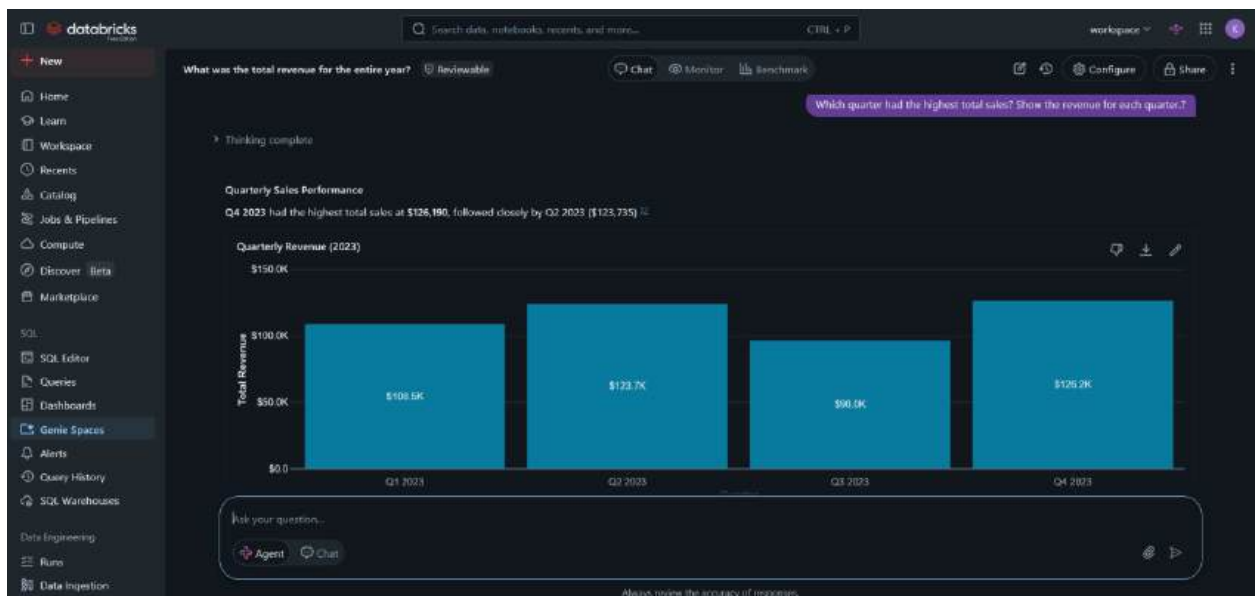
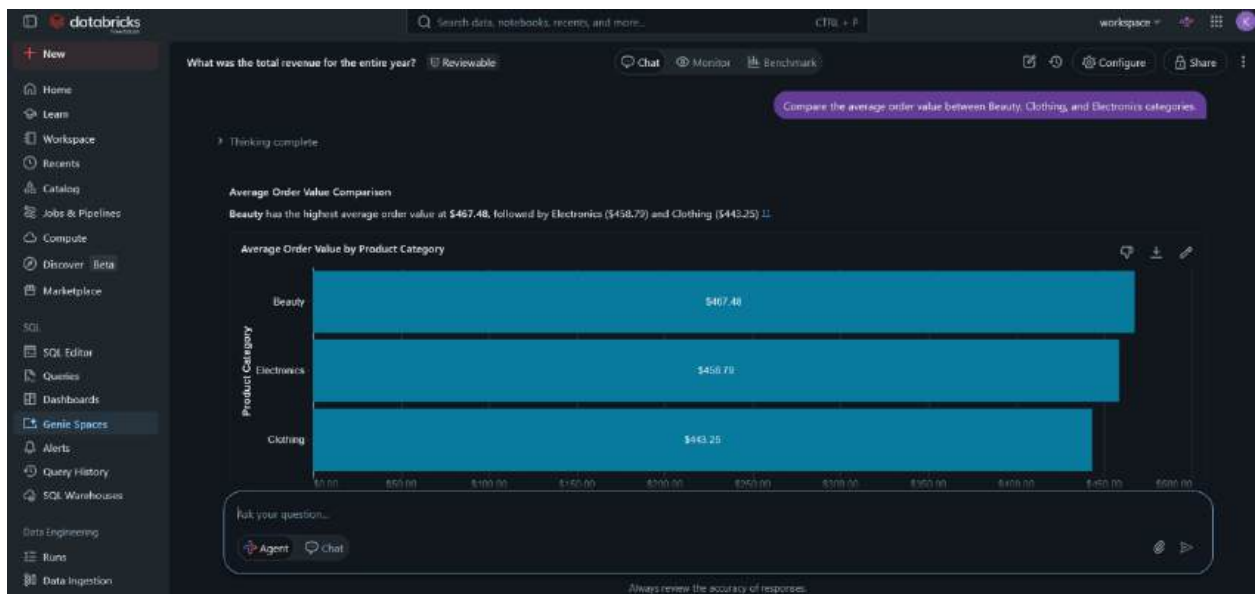
4. What is the average age of our customers? Is there a difference in spending between Male and Female customers?
5. How many high-value transactions (above \$1000) were there, and what was their total revenue?
6. Compare the average order value between Beauty, Clothing, and Electronics categories.
7. Which quarter had the highest total sales? Show the revenue for each quarter.
8. Who were the top 3 best-selling product categories by quantity sold?
9. Did younger customers (under 30) spend more or less on average compared to older customers?
10. What percentage of total revenue came from high-value transactions?

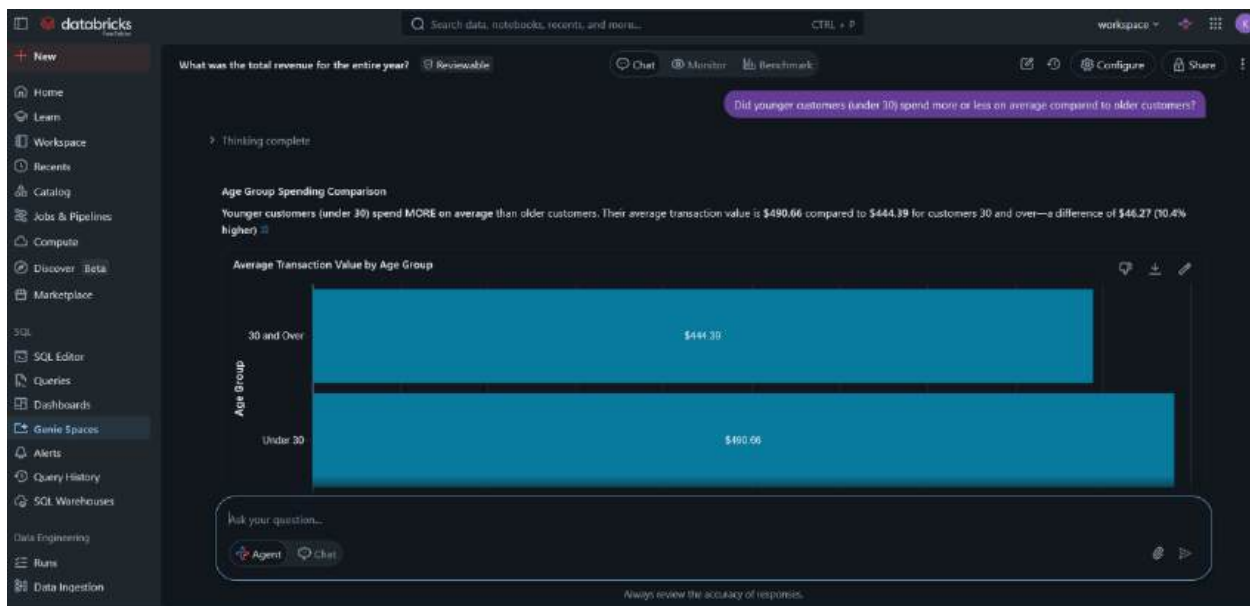
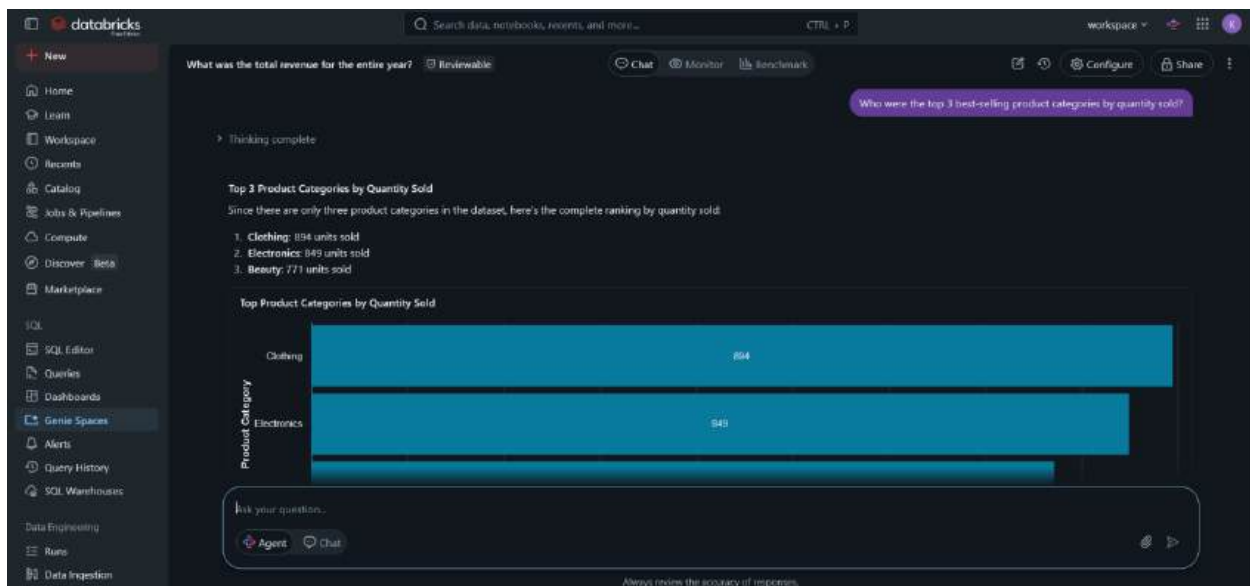
Screenshoot :

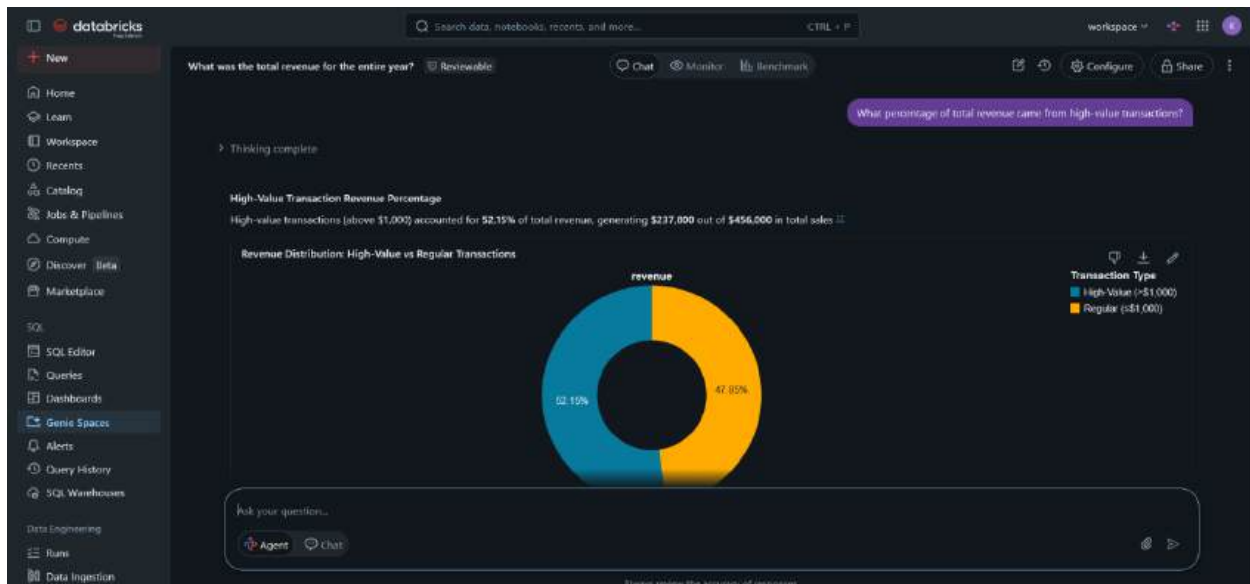












7. VALIDATE 3

In this step, I chose three questions to validate the Retail Sales Data Agent by testing it with 3 main important questions. The agent's responses were cross-checked using SQL queries.

Validation Results:

1. Question: What was the total revenue for the entire year?

Agent Answer: **\$456,000**

Below is my sql query i used to validate the agent's answer

```

47  YEAR(Date) AS Year,
48  MONTH(Date) AS Month,
49  QUARTER(Date) AS Quarter,
50  DAYOFWEEK(Date) AS Day_of_Week,
51  'Total Amount' / Quantity AS Revenue_per_Unit,
52  CASE
53  WHEN 'Total Amount' > 1000 THEN 'Yes' ELSE 'No' END AS High_Value_Transaction
54
55
56  FROM retail2.schema_retail.retail_sales_dataset;
57
58  --Validation --
59  -- 3 Validation Questions --
60
61  1.What was the total revenue for the entire year
62  SELECT SUM('Total Amount') AS Total_Revenue
63  FROM retail2.schema_retail.retail_sales_data_for_a_data_agent_improved;
64  --2.

```

Add parameter

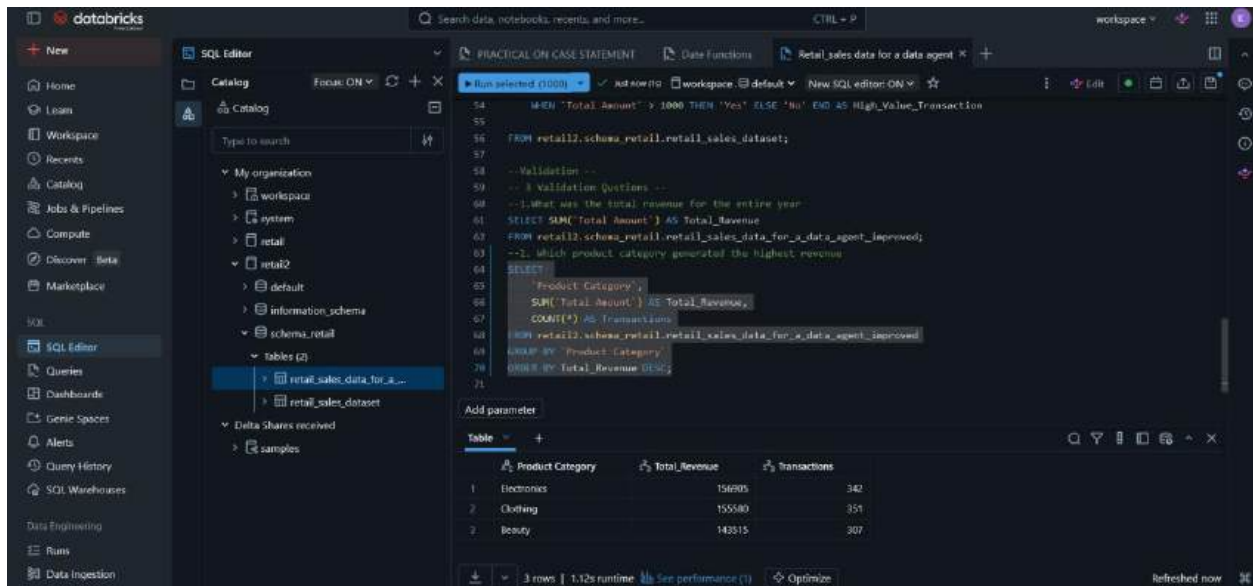
Table	Total_Revenue
1	456000

Validation: ☒ Its very much Accurate

2. Question: Which product category generated the highest revenue?

Agent Answer: **Electronics** generated the highest revenue at **\$156,905**

My sql query



The screenshot shows the Databricks SQL Editor interface. The left sidebar contains a navigation menu with options like Home, Workspace, Catalog, and SQL Editor. The main area displays a SQL query in a dark-themed editor. The query is a SELECT statement that filters for transactions with a total amount greater than 1000 and then groups the results by product category to find the highest revenue. The results are shown in a table at the bottom of the editor.

```
54 WHEN 'Total_Amount' > 1000 THEN 'Yes' ELSE 'No' END AS High_Value_Transaction
55
56 FROM retail2.schema_retail.retail_sales_dataset;
57
58 -- Validation --
59 -- 1 Validation Questions --
60 --1. What was the total revenue for the entire year
61 SELECT SUM('Total_Amount') AS Total_Revenue
62 FROM retail2.schema_retail.retail_sales_data_for_a_data_agent_improved;
63 --2. Which product category generated the highest revenue
64 SELECT
65     'Product Category',
66     SUM('Total_Amount') AS Total_Revenue,
67     COUNT(*) AS Transactions
68 FROM retail2.schema_retail.retail_sales_data_for_a_data_agent_improved
69 GROUP BY 'Product Category
70 ORDER BY Total_Revenue DESC;
71
```

Product Category	Total_Revenue	Transactions
Electronics	156905	342
Clothing	155880	351
Beauty	143515	307

Validation: ☒ Accurate

3. Question: What is the average age of customers and spending difference between Male and Female?

Agent Answer: Customer Age and Gender Spending Analysis

The average customer age is **41.4 years**, with customers ranging from 18 to 64 years old 5

Gender Spending Comparison

There is **minimal difference** in spending between male and female customers:

My sql query:

.

Screenshot of the Databricks SQL Editor interface showing a SQL query and its results.

SQL Query:

```
--2. Which product category generated the highest revenue
SELECT
  "Product Category",
  SUM("Total Amount") AS Total_Revenue,
  COUNT(*) AS Transactions
FROM retail2.schema_retail.retail_sales_data_for_a_data_agent_improved
GROUP BY "Product Category"
ORDER BY Total_Revenue DESC;

-- 3. What is the average age of customers and is there a big difference in spending between Male And Female --
SELECT
  AVG(Age) AS Average_Age,
  Gender,
  SUM("Total Amount") AS Total_Revenue,
  AVG("Total Amount") AS Avg_Order_Value,
  COUNT(*) AS Transactions
FROM retail2.schema_retail.retail_sales_data_for_a_data_agent_improved
GROUP BY Gender;
```

Results Table:

1.2 Average Age	1.2 Gender	1.2 Total Revenue	1.2 Avg_Order_Value	1.2 Transactions
41.42857142857143	Male	223160	455.42857142857144	490
41.35686274509804	Female	232040	456.54901960784315	510

2 rows | 0.58s runtime | See performance (T) | Optimize

Validation:  Accurate

-----THE END-----

SUBMITING ON GITHUB

Retail sales - Databricks

logins@gray-prod/retail_sales

dbx-d763822f-cc71.cloud.databricks.com/browse/folders/4035349424883147?as=74746531816473198;openGlt=4035949424883146;gltFile=Rela...

Retail_sales

Send feedback

Branch: main

Create Branch

66f500e

↓ Pull

Changes

Settings

1 changed file

Retail_Sales_Data_Agent_KAGSO_MATENCHI_QUERY.. A

Uploading A SQL CODE

Description (optional)

Commit & Push

Retail_Sales_Data_Agent_KAGSO_MATENCHI_QUERY.sql

Code

Raw file

Out 1

```
+ --Getting an overview of the data
+ SELECT +
+ FROM retail2.schema_retail.retail_sales_dataset;
+
+ -- Overall Summary
+ SELECT
+ COUNT(*) as total_transactions,
+ COUNT(DISTINCT 'Customer ID') as unique_customers,
+ MIN(Date) as first_date,
+ MAX(Date) as last_date,
+ SUM('Total Amount') as total_revenue,
+ AVG('Total Amount') as avg_order_value,
+ AVG(Quantity) as avg_quantity
+ FROM retail2.schema_retail.retail_sales_dataset;
+
+ -- By Product Category
+ SELECT
+ 'Product Category',
+ COUNT(*) as transactions,
+ SUM('Total Amount') as revenue,
+ AVG('Total Amount') as avg_order_value,
+ AVG(Quantity) as avg_qty
+ FROM retail2.schema_retail.retail_sales_dataset
```